LINEAMIENTO PARA EL VERSIONAMIENTO DEL CODIGO FUENTE DE ALFA

TABLA DE CONTENIDO:

1 OBJETIVO 3

2 INTRODUCCIÓN 3

3 ROLES Y RESPONSABILIDADES 3

3.1 Coordinador de desarrollo: 3

3.2 Líder técnico 3

3.3 Desarrollador 4

3.4 Devops 4

4 ¿QUÉ ES EL CONTROL DE VERSIONES? 4

5 DEFINICIONES DEL SISTEMAS DE CONTROL DE VERSIONES 5

5.1 Tipo de sistema de control de versiones 5

5.2 Nombramiento de repositorios 6

<repository type>-<project name>-<complement> 6

5.3 Gestión de repositorios en GitHub 7

5.3.1 Creación de Organizaciones 7

5.3.2 Definir Equipos y Roles 7

5.3.3 Configurar Repositorios y Permisos 7

5.3.4 Políticas de Aprobación para Pull Requests (PR) 7

5.3.5 Protección de Ramas 8

5.3.6 Requerimientos de Estado de Pruebas 8

5.4 Estrategia de branching 8

5.4.1 Git Flow 9

PROCESO 10

5.4.2 Integración selectiva de features 11

PROCESO 11

5.5 Reglas básicas de ramas 12

5.6 Nombramiento de ramas incluyendo el número de ticket jira (u otra herramienta de gestión de proyectos). 12

5.7 Creación de tags 13

5.8 Estándares de commit 13

5.8.1 Estructura de Commit 13

<type> 13

<ticket> 14

<subject> 14

5.8.2 Consideraciones para tomar en cuenta al momento de crear un commit 14

6 CONTROLES 14

1 OBJETIVO

Establecer los lineamientos que se deben realizar para el correcto versionamiento del Código fuente y de objetos de base de datos de las aplicaciones de Seguros Alfa.

2 INTRODUCCIÓN

A lo largo de la vida del desarrollo de un software es muy habitual que se vayan produciendo cambios en el código, realizados por todos los miembros que trabajan en el proyecto. Para un buen funcionamiento y que todos tengan en todo momento las modificaciones que se han llevado a cabo, es necesario contar con una buena herramienta que permita la gestión del control de versiones.

3 ROLES Y RESPONSABILIDADES

3.1 Coordinador de desarrollo:

Funcionario del área de tecnología de ALFA encargado de evaluar la viabilidad del requerimiento solicitado, de recibir cada versión entregada por los desarrolladores, haciendo las revisiones pertinentes al cumplimiento de las políticas definidas por ALFA.

Sus responsabilidades son las siguientes:

- Entrega del lineamiento de versionamiento de código fuente de alfa, al líder técnico, gestor del proyecto o personas que corresponda, para dar cumplimiento al mismo.
- Sesiones periódicas con al líder técnico, gestor del proyecto o personas que corresponda, para validación del cumplimiento del lineamiento de versionamiento de código fuente de alfa.
- Revisiones periódicas para validar el cumplimiento del lineamiento del versionamiento del Código fuente de alfa.

3.2 Líder técnico

Persona que analiza técnicamente las necesidades de negocio, impactos e integraciones con las demás aplicaciones de alfa y genera el documento de especificación técnica el cual es insumo para el inicio del desarrollo del requerimiento solicitado.

Sus responsabilidades son las siguientes:

- Entendimiento del lineamiento de versionamiento de código fuente de alfa.

- Asegurar el cumplimiento del lineamiento de versionamiento de código fuente de alfa.
- Revisión y aprobación de los PR, realizados por los desarrolladores.

3.3 Desarrollador

Persona encardada de realizar el análisis del desarrollo, dar solución del requerimiento solicitado y los defectos que puedas llegar a surgir al momento de la entrega del software.

Sus responsabilidades son las siguientes:

- Cumplir con el proceso de lineamiento de versionamiento de código fuente de alfa.
- Entrega de la documentación correspondiente al momento de hacer la solicitud de PR, para ser revisada por el líder técnico.

3.4 Devops

Persona encardada de realizar la creación de los proyectos con las ramas correspondientes, los usuarios con sus respectivos permisos y asegurar el funcionamiento de la herramienta GitHub.

Sus responsabilidades son las siguientes:

- Configurar en el repositorio los proyectos y los usuarios.
- Velar por el correcto funcionamiento de la herramienta GitHub.

4 ¿QUÉ ES EL CONTROL DE VERSIONES?

Uno de los sueños de todo el mundo es poder volver al pasado para solucionar algún tipo de problema que tengamos en el presente. Por desgracia, estos viajes temporales no son posibles en la vida real, pero al menos sí que es posible hacerlo en el desarrollo de software gracias al control de versiones.

Un sistema de control de versiones es una herramienta capaz de registrar todos los cambios que se realizan en uno o más proyectos, guardando a su vez versiones anteriores del proyecto, versiones a las que podemos acudir al no funcionar la forma correcta la aplicación.

El uso de control de versiones no es algo nuevo, sino que se viene utilizando desde hace muchos años, aunque no de la forma en la que se puede hacer en la actualidad, y es que todo el mundo, alguna vez ante un cambio en el código de un proyecto, ha sacado copia del archivo en un directorio para volver al estado anterior en caso de que

fallen los cambios. Ésta precisamente la principal ventaja que ofrecen los sistemas de control de versiones actuales.

Además de poder volver a un estado anterior, los actuales sistemas de versiones también ofrecen la posibilidad de comparar los cambios realizados a lo largo del tiempo, ver quién modificó algo en el proyecto que pueda estar causando problemas o el momento exacto en el que alguien introdujo un error en el código.

5 DEFINICIONES DEL SISTEMAS DE CONTROL DE VERSIONES

A continuación, se presentan algunas definiciones del sistema de control de versiones y la forma de trabajo implementada en Seguros Alfa. Este capítulo proporciona una comprensión clara del uso del sistema de control de versiones centralizado basado en Git y la plataforma GitHub, destacando las prácticas y procedimientos clave que se siguen para asegurar una gestión eficiente y colaborativa de los proyectos de desarrollo.

5.1 Tipo de sistema de control de versiones

En Seguros Alfa, se utiliza un sistema de control de versiones centralizado basado en Git. Este sistema permite llevar un seguimiento detallado de los cambios realizados en los proyectos, facilitando la colaboración y la integración continua.

Git es una elección debido a su robustez y flexibilidad, permitiendo gestionar eficazmente los desarrollos sin importar la complejidad o el tamaño del proyecto. Git ofrece la capacidad de crear ramas, fusionar cambios y mantener un historial completo de todas las modificaciones realizadas en el código. Esta característica es esencial para garantizar la integridad y calidad de los productos.

Como plataforma de alojamiento y gestión de los repositorios Git, Seguros Alfa ha seleccionado GitHub, ya que proporciona una interfaz amigable y una serie de herramientas adicionales que optimizan el flujo de trabajo, tales como la revisión de código, la gestión de incidencias y la integración con otros servicios utilizados en el ciclo de desarrollo.

El uso de Git, combinado con GitHub, permite a Seguros Alfa mantener un control riguroso sobre las versiones, fomentar la colaboración entre equipos y asegurar que todos los cambios se integren de manera ordenada y segura en los proyectos.

5.2 Nombramiento de repositorios

Debido a la gran cantidad de repositorios en Seguros Alfa, es necesario definir una estrategia clara para su nomenclatura. Esta estrategia tiene como objetivo garantizar que los repositorios puedan ser fácilmente localizados y reconocidos, facilitando su gestión y acceso.

Para el nombramiento de los repositorios, se utilizará el formato kebab case, que consiste en unir las palabras mediante guiones bajos (-). La estructura del nombre del repositorio seguirá el siguiente patrón, dependiendo del tipo de arquitectura y el propósito del repositorio:

<repository type>-<project name>-<complement>

- **Repository type**: Identifica la naturaleza del repositorio (por ejemplo, "msa" para microservicio de aplicación, "lib" para bibliotecas, "iac" para infraestructura, etc.).
- **Project name**: Describe el nombre principal del proyecto asociado.
- **Complement**: Incluye información adicional relevante (por ejemplo: En caso de aplicaciones contenerizadas usar el nombre del microservicio).

A continuación, se agrega tabla con los diferentes tipos de repositorios y ejemplos del estándar de nombrado:

Tipo	Descripción
lib	Librerías
web	Interfaz web
msa	Microservicio
iac	Infraestructura como código
db	Base de datos
sc	Infraestructura sin servidor
doc	Documentación
soa	Arquitectura orientada a servicios
mob	móvil
data	Proceso de datos
ma	Arquitectura monolítica

***Tabla 1**: Tipos de repositorios en Seguros Alfa*

Ejemplo: msa-sipren-calculations

5.3 Gestión de repositorios en GitHub

Para gestionar el código fuente de manera eficiente y segura, es fundamental organizar los equipos en Git. A continuación, se presenta una guía para estructurar la organización por equipos en Git.

5.3.1 Creación de Organizaciones

En GitHub se deben crear estructuras organizacionales permite gestionar múltiples repositorios, equipos y permisos desde un solo lugar.

5.3.2 Definir Equipos y Roles

Organizar a los miembros del equipo en grupos con permisos específicos. Los equipos suelen estar basados en roles o funciones dentro del proyecto.

Definición para los Equipos: Incluyen los desarrolladores que trabajan en un equipo específico. Ejemplo: dev-team.

5.3.3 Configurar Repositorios y Permisos

Cada equipo puede tener acceso a diferentes repositorios con distintos niveles de permisos. Configurar los permisos de acuerdo con la necesidad de cada equipo:

- **Acceso de Solo Lectura**: Para equipos que necesitan revisar el código, pero no modificarlo.
- **Acceso de Lectura y Escritura**: Para equipos que desarrollan y necesitan hacer cambios en el código.
- **Acceso Administrativo**: Para equipos que gestionan repositorios y la configuración de proyectos.

5.3.4 Políticas de Aprobación para Pull Requests (PR)

Las políticas de aprobación para pull requests son cruciales para garantizar que el código integrado en la base de código principal sea revisado, probado y aprobado adecuadamente.

- **Revisores Obligatorios**: Establecer que ciertas personas o equipos deben revisar y aprobar una PR antes de que pueda ser fusionada. Esto asegura que el código sea revisado por expertos en la materia o por miembros del equipo con el conocimiento necesario.

- **Número Mínimo de Revisores**: Especificar un número mínimo de revisores necesarios para aprobar una PR. Por ejemplo, requerir que al menos dos revisores aprueben la PR antes de fusionarla.

5.3.5 Protección de Ramas

- **Protección de la Rama Principal**: Proteger ramas específicas (main, quality, develop) para que no se puedan fusionar cambios sin cumplir con ciertas políticas, como aprobaciones requeridas y pruebas exitosas.
- **Requerir Revisiones Antes de Fusionar**: Configurar la protección de ramas para exigir que se realicen revisiones y aprobaciones antes de permitir la fusión de PR.

5.3.6 Requerimientos de Estado de Pruebas

- **Integración de CI/CD**: Configurar GitHub para que todas las pruebas automáticas (como las ejecutadas por CI/CD) pasen antes de que una PR pueda ser fusionada. Esto ayuda a asegurar que los cambios no rompan la funcionalidad existente.
- **Revisión de Estado de Pruebas**: Asegurar que los estados de las pruebas sean exitosos o cumplan ciertos criterios antes de que se pueda aprobar la PR.

5.4 Estrategia de branching

Una estrategia de branching es una serie de reglas que aplica un equipo de desarrollo de software de alfa cuando necesita escribir un código para incorporar una nueva funcionalidad o hacer una corrección, fusionarlo y enviarlo al repositorio donde se encuentra alojado el resto del código del software en uso.

La estrategia de branching debe buscar:

- Permitir a los desarrolladores optimizar su productividad.
- Habilitar el desarrollo en paralelo.
- Permitir un conjunto de releases estructurados y planificados.
- Proporcionar una ruta clara para promover los cambios del software al entorno de producción.
- Evolucionar y adaptarse a los cambios que se realizan a diario en el código.

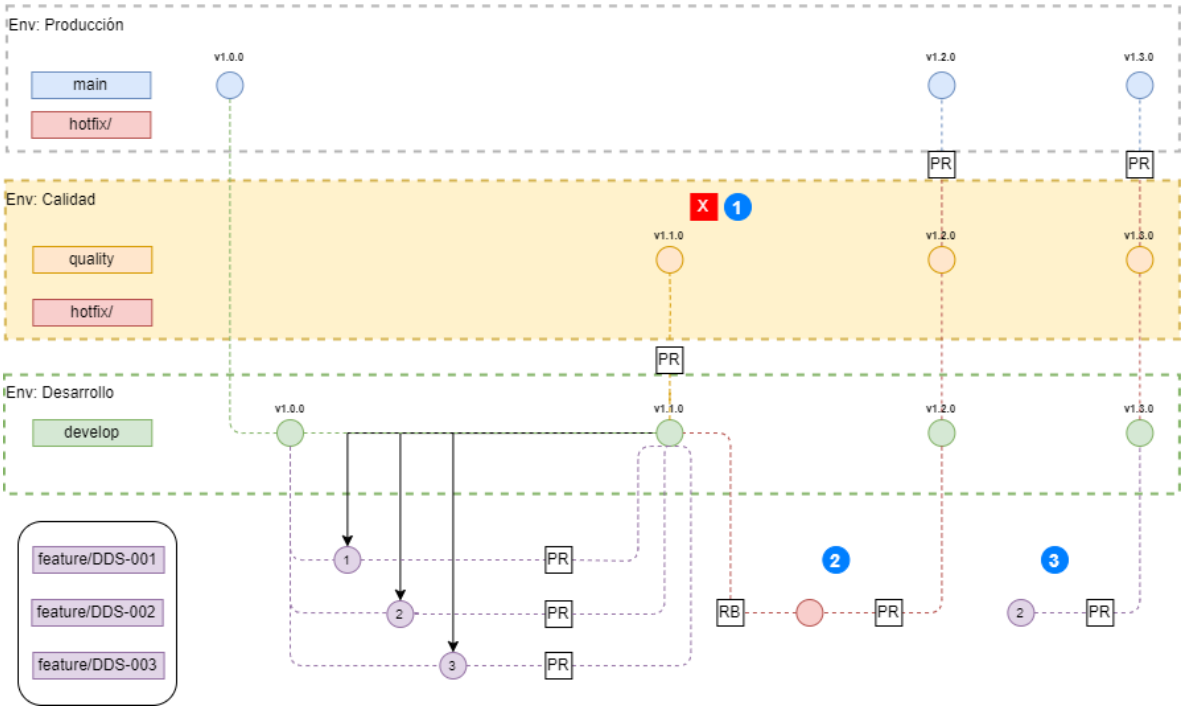
Dado que las dinámicas de los diferentes proyectos y los requerimientos de negocio asociados varían considerablemente, cada equipo en Seguros Alfa puede contar con una

forma de trabajo con Git diferente. Estas diferencias reflejan las necesidades específicas y características únicas de cada proyecto y equipo. No obstante, para garantizar un marco de trabajo coherente y eficiente, a continuación, se presentan los lineamientos generales que deben seguirse en las estrategias actuales de uso de Git.

Estos lineamientos están diseñados para proporcionar una base común que facilite la colaboración entre equipos, asegure la calidad del código y mantenga un historial claro y organizado de los cambios realizados en todos los proyectos de Seguros Alfa. Al adherirse a estas directrices, los equipos podrán gestionar sus repositorios de manera más efectiva y alinearse mejor con las prácticas recomendadas de la organización. Se definió en seguros ALFA utilizar dos estrategias de branching, y son las siguientes:

5.4.1 Git Flow

Gitflow es un modelo de gestión de ramas de Git en el que se utilizan ramas de efímeras y ramas fijas.



Nota: Para el cumplimiento de este flujo es responsabilidad del desarrollador mantener actualizada su rama (Remoto/Local) con develop.
Cada rama será integrada por Pull Request en la rama de develop, se debe tener en cuenta que:

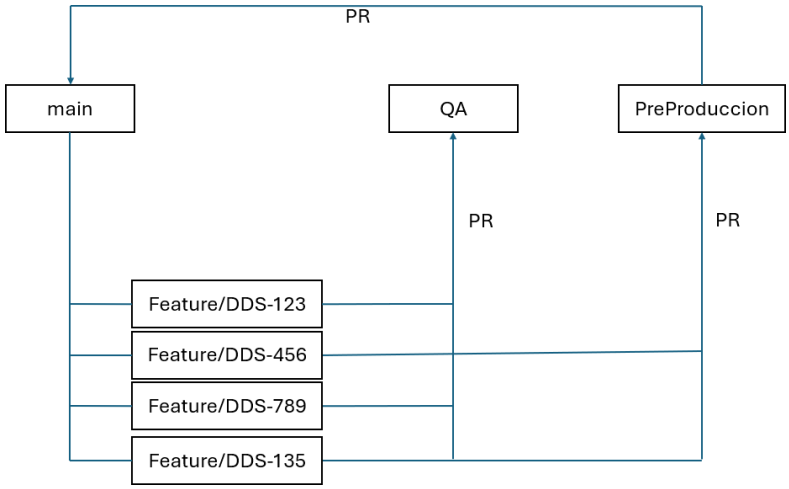
- Todas las historias planeadas deben pasar por el sprint para generar el tag.

- Cada tag generado compilará una imagen de desarrollo, el cual será desplegado en cada ambiente.
- En caso de no alcanzar a completar la historia, esa funcionalidad pasa a ser integrada en el siguiente sprint, actualizado el tag.
- Si ocurre un error en algún ambiente con ese tag generado, se debe sacar una rama para corregir el error, se integra a la rama develop, se vuelve a generar el tag y se elimina la rama fix.

PROCESO

1. Planear correctamente las Historias de usuarios (HU) que se van a pasar para cada sprint.
2. Para cada HU, el desarrollador que la tiene asignada deberá crear una rama "feature/Nombre_HU" en la que va a trabajar, esta rama debe partir siempre de develop.
3. Al finalizar el desarrollo de la HU, el desarrollador deberá crear un Pull Request (PR) de su rama con la de develop para su revisión.
4. Los PR creados deben ser validados por el líder técnico para su aceptación e integración con develop.
5. En caso de encontrar una inconsistencia con el desarrollo, el líder técnico rechazará el PR y dejará las notas correspondientes al rechazo del mismos.
6. El desarrollador debe ajustar los cambios y repetir el proceso del PR.
7. Una vez los cambios sean integrados a la rama develop, se procederá a crear el tag con la versión correspondiente.
8. El tag lo debe crear bien sea el líder técnico o el Devops para llevar correctamente las versiones.
9. Cuando se crea el tag, se desplegará con Jenkins en el ambiente correspondiente.
10. Para despliegues en otros ambientes se debe crear pull request de la rama asociada al ambiente previo hacia la rama asociada al siguiente ambiente según el esquema de ambientes del proyecto.
11. En el caso de que se requieran quitar funcionalidades se deben seguir los siguientes pasos marcados en el diagrama:
    1. **Sin aprobación**: Código que, por decisiones de negocio o pruebas, se identifica que no va a pasará y necesita cambios antes de pasar a los siguientes ambientes.
    2. **RB en develop**: Se revierte el PR integrado (#2) que se identifica para correcciones. El revert crea un PR sin la integración (#2) para ser integrado en develop nuevamente.
    3. **Continuación**: Se continua con el desarrollo de los incidentes para integrarse posteriormente en la rama develop.

5.4.2 Integración selectiva de features



PROCESO

1. Se debe crear por parte del coordinador o líder técnico, una rama por cada requerimiento de desarrollo que se vaya a trabajar, para ser utilizada por el desarrollador asignado (Feature/DDS-123).
2. Una vez se terminen los ajustes por parte del desarrollador, el mismo debe hacer el pull de su local, a la rama creada para dicho fin en el repositorio.
3. El desarrollador debe realizar un Pull Request (PR) hacia la rama de QA.
4. Los PR creados deben ser validados por el líder técnico para su aceptación e integración con la rama de QA.
5. En caso de encontrar una inconsistencia con el desarrollo, el líder técnico rechazará el Pull Request y dejará las notas correspondientes al rechazo del mismos.
6. El desarrollador debe ajustar los cambios y repetir el proceso del Pull Request.
7. Una vez los cambios sean integrados a la rama QA, se procederá a crear el artefacto para por parte del desarrollador para desplegarlo en dicho ambiente.
8. Una vez culminadas las pruebas y se cuente con la aprobación de estas. Se debe hacer por parte del desarrollador o líder técnico del proyecto un Pull Request desde la rama del desarrollo (Feature/DDS-123) hacia la rama de pre-produccion.
9. El equipo de demanda va a realizar la generación del artefacto para paso a producción desde la rama de pe-produccion.
10. pasado 7 días de estar en producción o dependiendo del requerimiento, se procede por parte de demanda a realizar un Pull Request desde la rama pre-produccion hacia la rama main.

5.5 Reglas básicas de ramas

1. **Minúsculas y con guiones separados:** Usa minúsculas para los nombres de las ramas y utiliza guiones para separar las palabras. Por ejemplo, *feature/nuevo-login-bugfix/estilo-cabecera.*
2. **Caracteres alfanuméricos:** Utilice solo caracteres alfanuméricos (a-z, 0–9) y guiones. Evite la puntuación, los espacios, los guiones bajos o cualquier carácter no alfanumérico.
3. **Sin guiones continuos**: No utilice guiones continuos. puede ser confuso y difícil de leer. *feature--nuevo-login.*
4. **Sin guiones finales:** No terminar el nombre de tu rama con un guión. Por ejemplo, no es una buena práctica *feature-nuevo-login-*.
5. **Descriptivo:** El nombre debe ser descriptivo, conciso y consistente, idealmente reflejando el trabajo realizado en la rama.

5.6 Nombramiento de ramas incluyendo el número de ticket jira (u otra herramienta de gestión de proyectos).

En algunos flujos de trabajo, especialmente en equipos más grandes, es habitual incluir el número de ticket de una herramienta de gestión de proyectos como Jira. Esto facilita el seguimiento del trabajo realizado en un ticket específico. Por ejemplo, si está trabajando en un ticket numerado "DDS-123" para agregar un nuevo sistema de inicio de sesión, el nombre de la sucursal podría ser feature/DDS-123.

1. **Feature Branches:** Se utilizan para nuevas funciones o mejoras. Ejemplo: *feature/<nombre-feature> feature/*DDS-123.
2. **Bug Fixes:** Se utilizan para resolver errores o problemas. Ejemplo: *bugfix/<ticket-bug> bugfix/* DDS-123*.*
3. **Hotfixes:** Se utilizan para correcciones críticas que deben abordarse de inmediato. Ejemplo: *hotfix/<ticket-correccion> hotfix/* DDS-123*.*
4. **Release Branches:** Se utilizan para preparar una nueva versión. Ejemplo: *release/1.2.0 (major, minor, patch).*
5. **Documentation Branches**: Se utilizan para escribir, actualizar o corregir documentación. Ejemplo: docs/<*descripcion-documentacion*> docs/api-endpoints.
6. **Experimental Branches:** Se utilizan para trabajos experimentales o spike que no se pueden fusionar. Ejemplo: *experiment/<nombre-experimento> experiment/nueva-arquitectura.*
7. **Chore Branches:** Se utilizan para tareas rutinarias como la actualización de dependencias o la refactorización. Ejemplo: *chore/<tarea> chore/actualizar-dependencias.*

5.7 Creación de tags

1. **Major(1):** El número principal de la versión, se incrementa cuando se realizan cambios incompatibles en la API o se introducen cambios importantes que pueden requerir ajustes significativos en el código.
2. **Minor(2):** El número menor de la versión, se incrementa cuando se agregan nuevas funcionalidades de una manera compatible hacia atrás, lo que significa que no se deben agregar cambios significativos en el código existente para utilizar esas nuevas funcionalidades.
3. **Patch(3):** El número de parche de a versión, se incrementa cuando se realizan correcciones de errores o cambios menores que son compatibles hacia atrás, lo que significa que no se introducen nuevas funcionalidades no cambios incompatibles.

5.8 Estándares de commit

El estándar de commit es la especificación o convención simple que se añade por encima de los commits convencionales, es decir nos indica la forma en la que debemos estructurar nuestros commits en Seguros Alfa. Esto permite tener mensajes más legibles que son fáciles de seguir cuando se busca en el historial del Proyecto.

5.8.1 Estructura de Commit

A continuación, conoceremos la estructura de un commit, así como la estructura que debe tener el header alineado a los estándares de Seguros Alfa.

Para hacer uso del estándar de commit debe seguir la siguiente estructura:

<type>-<ticket>-<subject>

<type>

A continuación, vamos a listar los tipos de commit:

- **ci**: Cambios en archivos de CI (DevSecOps).
- **docs**: Cambios en documentación.
- **feat**: Agrega una nueva funcionalidad.
- **fix/hotfix/patch**: Corrección de errores, aplica solo para la rama fix.
- **perf**: Cambio que mejora el rendimiento de la aplicación, pero sin cambios funcionales.
- **refactor**: Cambio que refactoriza el código, pero no agrega o modifica el negocio.
- n: mejoras/reescritura de features existentes, no agrega un cambio grande a lo que actualmente tiene. (i.e. cambiar estados locales usando componente conectados)

- **test**: Agrega pruebas unitarias faltantes o correcciones de pruebas existentes.
- **style**: Cambios que no afectan funcionalidades, usado para formatear código, linters, etc.
- **chore:** Se usa para cambios en configuraciones (dependencias externas, empaquetadores, etc) y cualquier otra tarea que no encaja con los otros tipos de commit. También se utiliza para un first o initial commit del repositorio o trabajos de migración donde no tiene sentido dividir los commits ya que simbolizan un único entregable.
- **revert**: deshacer los cambios efectuados.

<ticket>

identificación correspondiente a cada dirección, demanda (número de JIRA), célula (Número de JIRA y nombre de la célula), operaciones (Número de incidente o jira o requerimiento etc), proyecto (número de control interno y nombre del proyecto).

<subject>

Mensaje corto con la descripción de los cambios aplicados.

5.8.2 Consideraciones para tomar en cuenta al momento de crear un commit

- El subject no debe exceder de los 72 caracteres.
- El ticket es obligatorio y va en mayúsculas.
- El subject debe ser en minúsculas.
- El tiempo del mensaje debe ser en presente simple.
- El mensaje debe ser entendible y concise.
- Configurar Git incluyendo su usuario entregado por Seguros Alfa y email corporativo. Ejemplo:

  git config --global user.name "javmorales"
  git config --global user.email javmorales@segurosalfa.com.co

6 CONTROLES

- Por parte del líder técnico asignado o la coordinación de desarrollo, al momento de revisar y aprobar el Pull Request, debe validar que lo entregado por el desarrollador está cumpliendo con las buenas prácticas de desarrollo, que no están agregando código quemado en la implementación, etc.
- Revisión periódica por parte de la coordinación de desarrollo para validar el cumplimento del lineamiento definido para el versionamiento del código fuente.